

```
[....]

completed successfully (2868ms)
```

If everything is OK, we will receive a `completed successfully` message (as seen above); now we are ready to manage continuous backup.

Managing base backups

As we previously mentioned, `pgbackrest` is able to handle full backups, differential backups, and incremental backups with a simple command-line statement.

To create a full base backup, we can do this:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 --type=full backup
```

When we press the *Enter* key on the keyboard, if everything is OK, we get this message:

```
2023-07-11 08:42:58.125 P00    INFO: expire command end: completed
successfully (46ms)
```

Now, if we want information about our repository, we can use the `info` command as follows:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 info
stanza: pg1
  status: ok
  cipher: aes-256-cbc

  db (current)
    wal archive min/max (15): 000000010000000000000001/000000010000000
00000004

    full backup: 20230711-084245F
      timestamp start/stop: 2023-07-11 08:42:45 / 2023-07-11
08:42:57
      wal start/stop: 000000010000000000000004 /
000000010000000000000004
      database size: 22.0MB, database backup size: 22.0MB
      repo1: backup set size: 2.9MB, backup size: 2.9MB
```

The `info` command tells us about WAL segments, the full backup start time, the original database size, and the repository backup size.

In a similar way, starting with this full backup, we can make an incremental backup:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 --type=incr backup

2023-07-11 08:44:35.816 P00    INFO: expire command end: completed
successfully (15ms)
```

We can also make a differential backup:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 --type=diff backup

2023-07-11 08:45:40.020 P00    INFO: expire command end: completed
successfully (32ms)
```

Now an info command will track the three backups:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 info
stanza: pg1
  status: ok
  cipher: aes-256-cbc

  db (current)
    wal archive min/max (15): 000000010000000000000001/0000000100000000
000000008

    full backup: 20230711-084245F
      timestamp start/stop: 2023-07-11 08:42:45 / 2023-07-11
08:42:57
      wal start/stop: 000000010000000000000004 /
000000010000000000000004
      database size: 22.0MB, database backup size: 22.0MB
      repo1: backup set size: 2.9MB, backup size: 2.9MB

    incr backup: 20230711-084245F_20230711-084431I
      timestamp start/stop: 2023-07-11 08:44:31 / 2023-07-11
08:44:35
      wal start/stop: 000000010000000000000006 /
000000010000000000000006
      database size: 22.0MB, database backup size: 8.3KB
      repo1: backup set size: 2.9MB, backup size: 496B
```

```

        backup reference list: 20230711-084245F

        diff backup: 20230711-084245F_20230711-084536D
        timestamp start/stop: 2023-07-11 08:45:36 / 2023-07-11
08:45:39
        wal start/stop: 000000010000000000000008 /
000000010000000000000008
        database size: 22.0MB, database backup size: 8.3KB
        repo1: backup set size: 2.9MB, backup size: 496B
        backup reference list: 20230711-084245F

```

As we have set `repo1-retention-full=2` on the `pgbackrest.conf` file, `pgbackrest` (after two backups) will delete the first full backup and its linked differential or incremental backups. For example, here's the execution of two full backups:

```

postgres@pgbackrest:~$ pgbackrest --stanza=pg1 --type=full backup

2023-07-11 08:48:39.866 P00    INFO: expire command end: completed
successfully (22ms)

postgres@pgbackrest:~$ pgbackrest --stanza=pg1 --type=full backup

2023-07-11 08:49:37.101 P00    INFO: expire command end: completed
successfully (34ms)

```

We will then have the following outcome:

```

postgres@pgbackrest:~$ pgbackrest --stanza=pg1 info
stanza: pg1
  status: ok
  cipher: aes-256-cbc

  db (current)
    wal archive min/max (15): 000000010000000000000001/000000010000000
00000000C

    full backup: 20230711-084830F
    timestamp start/stop: 2023-07-11 08:48:30 / 2023-07-11
08:48:39

```

```

        wal start/stop: 00000001000000000000000A /
00000001000000000000000A
        database size: 22.0MB, database backup size: 22.0MB
        repo1: backup set size: 2.9MB, backup size: 2.9MB

        full backup: 20230711-084928F
        timestamp start/stop: 2023-07-11 08:49:28 / 2023-07-11
08:49:36
        wal start/stop: 00000001000000000000000C /
00000001000000000000000C
        database size: 22.0MB, database backup size: 22.0MB
        repo1: backup set size: 2.9MB, backup size: 2.9MB

```

As we can see, the system has automatically deleted the first full backup and its related incremental and differential backups.

Managing PITR

In this section, we will look at how to restore a PostgreSQL cluster after a disaster.

To build an example, let's create a table on the PostgreSQL server:

```

postgres=# create table users (id integer, user_name text);
CREATE TABLE

```

And let's populate it with some data:

```

postgres=# insert into users select generate_
series(1,10000), 'user_' || generate_series(1,10000)::text;
INSERT 0 10000

```

Now let's see what time it is on the PostgreSQL server:

```

postgres=# select now();
          now
-----
2023-07-11 08:55:08.22447+00
(1 row)

```

Let's suppose that a disaster has happened after this point in time; for example, suppose that we dropped a table after this time:

```

postgres=# drop table users;

```

```
DROP TABLE
```

Now let's try to make a recovery at 2023-07-11 08:55:08, which is the time before the disaster happened. On the pg1 server, we need to stop the postgresql server:

```
# systemctl stop postgresql
```

Then we perform the pgbackrest restore command:

```
root@pg1:~# su - postgres
postgres@pg1:~$ pgbackrest --stanza=pg1 --delta --log-level-console=info
--type=time "--target=2023-07-11 08:55:08" restore

2023-07-11 08:57:39.803 P00    INFO: restore command end: completed
successfully (1905ms)
```

Now let's start the postgresql server as the root user:

```
# systemctl start postgresql
```

Then we check the postgresql log:

```
2023-07-11 08:59:06.844 P00    INFO: archive-get command end: completed
successfully (532ms)
2023-07-11 08:59:06.849 UTC [8786] LOG:  restored log file
"000000010000000000000000D" from archive
2023-07-11 08:59:06.898 UTC [8786] LOG:  consistent recovery state reached
at 0/C000138
2023-07-11 08:59:06.898 UTC [8783] LOG:  database system is ready to
accept read-only connections
2023-07-11 08:59:06.938 P00    INFO: archive-get command begin 2.46:
[000000010000000000000000E, pg_wal/RECOVERYXLOG] --exec-id=8797-9e31ebc7
--log-level-console=info --log-level-file=info --pg1-path=/var/lib/
postgresql/15/main --repo1-host=192.168.122.120 --repo1-host-port=22
--repo1-host-user=postgres --stanza=pg1
2023-07-11 08:59:07.274 P00    INFO: unable to find
000000010000000000000000E in the archive
2023-07-11 08:59:07.375 P00    INFO: archive-get command end: completed
successfully (440ms)
```

```

2023-07-11 08:59:07.379 UTC [8786] LOG:  recovery stopping before commit
of transaction 737, time 2023-07-11 08:55:13.072211+00
2023-07-11 08:59:07.379 UTC [8786] LOG:  pausing at the end of recovery

2023-07-11 08:59:07.379 UTC [8786] HINT:  Execute pg_wal_replay_resume()
to promote.

```

As we can see, to end our PITR procedure, PostgreSQL suggests we execute `pg_wal_replay_resume()`. So, let's go into the PostgreSQL environment and perform the following:

```

postgres=# select pg_wal_replay_resume();
 pg_wal_replay_resume
-----
(1 row)

```

Now if we go to check our database, `db1`, the `users` table is now present and the database is now in the state that it was in at `2020-05-30 16:23:38`:

```

postgres=# \d
          List of relations
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | users | table | postgres
(1 row)

```

Finally, we can execute this:

```

postgres=# select count(*) from users ;
 count
-----
 10000
(1 row)

```

We have now restored the situation that was present before the disaster.

As we have seen, managing continuous and PITR backups with `pgbackrest` is really simple. Continuous backups and PITR should never be missing in the setup of a complex production environment. This protects us from unwanted data deletions and gives us a “last resort” to use when both primary and replica servers are no longer available.

Migrating from MySQL/MariaDB to PostgreSQL using pgloader

In this section, we will see how to migrate a database from the MySQL/MariaDB world to the PostgreSQL world in a very simple way. The tool we will use is called pgloader. The references for further information on this section can be found at <https://pgloader.io>.

Two Docker containers are available for this section, one with a mariadb server inside and another with a postgresql server inside. The mariadb server, called mariadb-source, contains a copy of the forumdb database used in *Chapter 4*; the postgresql server called pg-destination contains an empty database called forumdb. Our goal will be to migrate all the contents of the forumdb database from the mariadb-source server to the postgresql-destination server using the pgloader tool.

Let's open two Bash terminals, and in the first one, let's execute:

```
chapter19$ bash run-pg-docker-mariadb.sh
```

Once the container has started, let's execute the statement below, using LearnPostgreSQL as the password:

```
root@mariadb-source:~# mysql -D forumdb -p
[...]
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input
statement.

MariaDB [forumdb]>
```

Now we are inside the forumdb database and we can make a query on it:

```
MariaDB [forumdb]> show tables;
+-----+
| Tables_in_forumdb |
+-----+
| categories         |
| j_posts_tags       |
| posts              |
```

```

| tags          |
| users         |
+-----+
5 rows in set (0.000 sec)

MariaDB [forumdb]> select * from categories;
+----+-----+-----+-----+
| pk | title                | description                |
+----+-----+-----+-----+
| 1  | Database              | Database related discussions |
| 2  | Unix                  | Unix and Linux discussions  |
| 3  | Programming Languages | All about programming languages |
+----+-----+-----+-----+
3 rows in set (0.002 sec)

MariaDB [forumdb]> select * from users;
+----+-----+-----+-----+
| pk | username | gecos      | email                |
+----+-----+-----+-----+
| 1  | fluca1978 | Luca Ferrari | fluca1978@gmail.com |
| 2  | sscotty71 | Enrico Pirozzi | sscptty71@gmail.com |
+----+-----+-----+-----+
2 rows in set (0.001 sec)

```

As we can see, the database is the same as we used in *Chapter 4, Basic Statements*.

Now let's go to the second Bash terminal and execute:

```
chapter19$ bash run-pg-docker-postgresql.sh
```

Once the container starts, let's execute:

```
postgres@pg-destination:~$ psql forumdb
```

As we can see below, our postgresql database is empty:

```

postgres@pg-destination:~$ psql forumdb
forumdb=# \d
Did not find any relations.

```

Now let's exit from the psql client and let's execute the pgloader command:

```
forumdb=# \q
ppostgres@pg-destination:~$ pgloader mysql://root:LearnPostgreSQL@mariadb-
source/forumdb pgsql://postgres@127.0.0.1/forumdb
```

table name	errors	rows	bytes	total time
-----	-----	-----	-----	-----
fetch meta data	0	22		0.104s
Create Schemas	0	0		0.000s
Create SQL Types	0	0		0.004s
Create tables	0	10		0.056s
Set Table OIDs	0	5		0.020s
-----	-----	-----	-----	-----
forumdb.categories	0	3	0.1 kB	0.020s
forumdb.users	0	2	0.1 kB	0.016s
forumdb.j_posts_tags	0	0		0.012s
forumdb.tags	0	0		0.016s
forumdb.posts	0	0		0.012s
-----	-----	-----	-----	-----
COPY Threads Completion	0	4		0.024s
Index Build Completion	0	11		0.048s
Create Indexes	0	11		0.092s
Reset Sequences	0	4		0.012s
Primary Keys	0	4		0.004s
Create Foreign Keys	0	6		0.004s
Create Triggers	0	0		0.000s
Set Search Path	0	1		0.000s
Install Comments	0	0		0.000s
-----	-----	-----	-----	-----
Total import time	0	5	0.2 kB	0.184s

Using the simple command above, we have migrated the mariadb forumdb database into the postgresql forumdb database. Now let's go to check if everything has been migrated; let's re-connect to forumdb on the postgresql server:

```
postgres@pg-destination:~$ psql forumdb
```

```

forumdb=# \dn
          List of schemas
  Name  | Owner
-----+-----
forumdb | postgres
public  | pg_database_owner
(2 rows)

forumdb=# \dt forumdb.*
          List of relations
 Schema | Name      | Type | Owner
-----+-----+-----+-----
forumdb | categories | table | postgres
forumdb | j_posts_tags | table | postgres
forumdb | posts      | table | postgres
forumdb | tags       | table | postgres
forumdb | users      | table | postgres
(5 rows)

```

As we can see above, the mariadb database called forumdb has been migrated to the PostgreSQL database called forumdb. pgloader automatically creates a schema called forumdb where we can find all the tables and data coming from the original mariadb database forumdb; now the only thing we need is to create a user, forumdb, that has all the permissions to use all the data that we have just imported:

```

forumdb=# create role forumdb with password 'LearnPostgreSQL' login;
CREATE ROLE

forumdb=# grant usage on schema forumdb to forumdb ;
GRANT

forumdb=# grant all on all tables in schema forumdb to forumdb ;
GRANT

forumdb=# \q

```